in your code. There's a special type of map—structs—that solves these problems.

Structs are maps, but they have additional features added to them. Structs are named maps that have predefined keys, required keys, and default values.

Let's model an SMS message using a struct. Create lib/structs/sms.ex and add the following code:

elixir_examples/lib/examples/structs/sms_1.ex

```
defmodule Examples.Structs.SmsBasic do
  defstruct [:from, :to, :body]
end
```

And then run it:

```
$ iex -S mix
iex> sms = %Examples.Structs.SmsBasic{}
%Examples.Structs.SmsBasic{body: nil, from: nil, to: nil}

iex> is_map(sms)
true

iex> is_struct(sms)
true

iex> sms = %Examples.Structs.SmsBasic{to: "111-222-3333"}
%Examples.Structs.SmsBasic{body: nil, from: nil, to: "111-222-3333"}

iex> sms = Map.put(sms, :body, "A text message")
%Examples.Structs.SmsBasic{
  body: "A text message",
  from: nil,
  to: "111-222-3333"
}

iex> sms = %Examples.Structs.SmsBasic{nope: true}
** (KeyError) key :nope not found
```

You initialized the struct by writing the name of the module between the % and { symbols. By default, all of the provided keys are nil. You can initialize the struct with data, just like a map, but the keys must be part of the defined set or you get an error. Once you have the struct initialized, you use Map functions to manipulate it.

The way that structs are defined means that you can only have a single struct defined in a single module. If you want multiple structs, you need to create multiple modules.

Let's take our struct a step further by implementing required and default-value keys. Rewrite the module to the following code: